



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	Ms Md Razia
Student Name	G. Rishitha
Roll Number	2520030101
Branch	CSE
Course Outcome	CO1 — Apply Binary Search Trees (BST) and self-balancing AVL Trees to construct, insert, delete, and search hierarchical datasets, demonstrating correct rotation operations and $O(\log n)$ performance guarantees in real-world file indexing and dictionary scenarios.

Case Study Topic: SkillBridge – Course Enrollment Database Indexing using AVL Tree.

Work Done Summary:

In this case study, we implemented a self-balancing AVL Tree structure for SkillBridge's core catalog management system. The platform processes large numbers of enrollments, requiring an efficient method to store and access courses using unique Course IDs as unique numerical identifiers. The AVL tree guarantees that regardless of the insertion order of course listings, the hierarchical tree structure remains balanced.

We mapped sequential and arbitrary Course IDs into the platform structure and monitored the runtime triggering of balancing rotations (LL, RR, LR, RL) to neutralize height variances. The system successfully executes high-frequency insertions, recursive lookups, record omissions, and sorted catalog displays via inorder traversals. Utilizing an AVL Tree reduces search and retrieval latencies from traditional linear arrays to a strict logarithmic time boundary, preventing processing performance bottlenecks during peak user traffic registration periods.

Implementation Details:

1. BST and AVL Tree node creation with course, skill, and certification records.
2. Height maintenance and balance factor calculation.
3. Skill/Course record insertion and search operations.
4. Skill/Course record deletion and update operations.
5. AVL balancing through rotations (LL, RR, LR, RL).
6. In-order traversal for verification and sorted record display.
7. Complexity analysis and performance evaluation of BST and AVL Trees.

Result: Screenshots / Output:

```
PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\DELL\OneDrive\Rishitha> java SkillBridgeAVL
Enter number of courses/certifications: 5
Enter IDs:
50
30
70
20
40
In-order Traversal: 20 30 40 50 70
Enter ID to search: 40
Record Found
Enter ID to delete: 30
After Deletion: 20 40 50 70
```

Result and complexity:

1. The BST and AVL Tree structures successfully managed course and certification records through insertion, deletion, and search operations.
2. AVL balancing was maintained using LL, RR, LR, and RL rotations whenever required.
3. In-order traversal displayed records in sorted order, verifying the correctness of the tree structure.
4. Search operations efficiently retrieved course and certification records using their unique IDs.
5. Time Complexity Bounds:
6. BST Search/Insertion/Deletion (Average): **$O(\log n)$**
7. AVL Search/Insertion/Deletion: **$O(\log n)$**
8. In-order Traversal: **$O(n)$**

GitHub Link:

<https://github.com/rishi07-lab/DSA-2-CASE-STUDIES>

Student Signature	Faculty Signature